

BOOKFORGE INSIGHT REPORT

업무 자동화 재설계

프로세스부터 다시 그려라

	EXECUTIVE SUMMARY	1
01	재설계 없는 자동화의 함정	2
	왜 자동화는 반복적으로 실패하는가	3
	핵심 논지: 결함까지 그대로 자동화된다는	3
	이 책이 제안하는 순서	4
02	현재 프로세스 진단	5
	있는 그대로 그리는 법	6
	매핑에서 드러나는 세 가지 낭비	7
03	재설계 4원칙	9
	비효율을 자동화하면 비효율의 속도만 빨라진다	10
	제거 — 이 단계는 왜 존재하는가	10
	통합 — 통합 가능한 승인은 없는가	11
	표준화 — 예외를 표준 경로로 흡수할 수 있는가	11
	자동화 — 남은 반복 작업만 자동화 대상인가	12
	순서를 건너뛰지 않는 것이 재설계의 전부다	13
04	자동화 기술 지형도	14
	데이터 레이어를 중심으로 만나는 네 기술	15
	화면을 대신 조작하는 기술, RPA	16
	흐름을 관리하는 워크플로우 엔진	16
	시스템과 시스템을 잇는 API 연동	17
	판단이 필요한 구간의 AI 에이전트	17
	하나의 요청이 지나가는 다섯 상태	18
05	파일럿에서 확산까지	19
	파일럿 대상을 고르는 기준	20
	병행 운영 — 두 흐름을 동시에 돌린다	21
	확산은 인접 프로세스부터	21
06	거버넌스와 오너십	22
	담당자 한 명에게 남은 로직	23
	예외는 어디로 가야 하는가	23
	변경은 누가 승인하는가	24
07	재설계 이후의 조직 역량	26
	프로젝트로 끝나면 왜 다시 낡는가	27
	다시 반복할 수 있는 세 가지 역량	27
	습관으로 남기 위한 방향	28

자동화의 성패는 도구 아닌 순서가 가른다

RPA·워크플로우 엔진·AI 에이전트부터 들인 조직은 몇 달 안에 같은 벽에 부딪힌다. 예외가 폭증하고 병목이 자리만 옮기며, 담당자가 떠나면 로직은 블랙박스가 된다. 도구를 바꿔도 결과는 달라지지 않는다. 결함 있는 프로세스 위에서는 새 도구가 같은 결함을 더 세련된 방식으로 반복할 뿐이기 때문이다. 원인은 도구가 아니라 순서다. 이 보고서는 진단, 재설계, 기술 선택, 파일럿, 거버넌스의 다섯 단계를 도구 선택보다 앞에 두는 실행 순서를 제안한다. 프로세스를 먼저 다시 그리고 나서 자동화를 얹으면, 자동화할 대상 자체가 줄고 남은 자동화는 오래간다.

1. 결함까지 그대로 자동화된다

결함 위에 도구를 얹으면 비효율이 반복되는 속도만 빨라진다. 불필요한 승인은 더 빨리 돌고 중복 입력은 더 자주 쌓인다. 도구 교체로는 바뀌지 않는다.

2. 진단은 실제 흐름을 그린다

담당자의 설명이 아닌 실제 처리 기록으로, 부서·역할 레인 위에 있는 그대로 그린다. 숨은 핸드오프·중복 입력·대기 구간은 이 지도에서만 드러난다.

3. 제거·통합·표준화가 먼저다

모든 단계에 존재 이유를 물어 제거하고, 흩어진 승인을 합치고, 예외를 표준 경로로 흡수한다. 세 관문을 통과한 반복 작업만 자동화 대상이 된다.

4. 기술은 작업 성격이 정한다

화면 반복은 RPA, 승인 흐름은 워크플로우 엔진, 시스템 연동은 API, 판단이 필요한 예외는 AI 에이전트가 맡는다. 한 기술로 전부를 덮으면 무리가 생긴다.

5. 검증 후 인접부터 넓힌다

빈도 높고 예외 적은 프로세스를 파일럿 삼아 병행 운영으로 검증한다. 전환 판단 기준을 미리 합의한 뒤, 인접 프로세스부터 단계적으로 확산한다.

6. 자동화 이후에도 주인이 필요하다

프로세스 오너·자동화 담당·예외 처리 창구·변경 승인권자 네 역할을 재설계 단계에서 함께 정한다. 주인 없는 자동화는 수작업으로 되돌아간다.

4

자동화 앞의 재설계 관문

1

확산 전 파일럿 검증

4

자동화 이후에 서야 하는 역할

01

재설계 없는 자동화의 함정

도구를 먼저 들이고 프로세스는 그대로 두는 자동화가 왜 반복적으로 실패하는지, 그리고 이 책이 제안하는 순서 — 프로세스를 먼저 다시 그리고 나서 자동화를 엮는 순서 — 를 소개한다.

왜 자동화는 반복적으로 실패하는가
핵심 논지: 결함까지 그대로 자동화된다는
이 책이 제안하는 순서

최근 여러 조직에서 RPA·워크플로우 툴·AI 에이전트를 잇달아 도입하고 있다. 반복 업무를 없애겠다는 목표는 분명하지만, 결과는 기대만큼 깔끔하지 않은 경우가 적지 않다. 도구를 들이는 순간에는 처리 속도가 눈에 띄게 빨라지는 듯 보이다가도, 몇 달이 지나면 예외 상황이 늘어나고, 담당자가 자리를 옮기면 자동화 로직을 아무도 손대지 못하는 상태로 남는다. 다수 조직에서 반복적으로 관찰되는 이 패턴은 도구의 성능 문제가 아니다. 자동화를 엮은 대상, 즉 프로세스 자체에 원인이 있다.

왜 자동화는 반복적으로 실패하는가

가장 먼저 나타나는 증상은 **예외 처리의 폭증**이다. 기존 프로세스는 오랫동안 사람이 재량으로 처리해 온 예외 경로를 암묵적으로 품고 있다. 이 재량을 그대로 둔 채 정형화된 규칙만 자동화하면, 규칙에 걸리지 않는 케이스가 자동화 바깥으로 계속 밀려난다. 시간이 지날수록 예외 처리 목록은 늘어나고, 결국 “자동화된 절차”와 “실제로 사람이 처리하는 절차”가 나란히 굴러가는 이중 구조가 만들어진다.

두 번째 증상은 **병목의 이동**이다. 특정 단계의 처리 속도만 자동화로 끌어올리면, 그 앞뒤에 있던 수작업 단계가 새로운 병목이 된다. 승인 대기가 그 전형이다. 신청 접수는 즉시 처리되지만 승인권자의 결재 속도는 그대로이므로, 전체 처리 시간은 거의 줄지 않고 병목의 위치만 바뀐다. 프로세스 전체를 보지 않고 한 구간만 자동화했을 때 반복적으로 나타나는 결과다.

세 번째는 **담당자 이탈 후의 유지보수 불가**다. 자동화 로직을 설계할 때 프로세스의 예외와 배경 지식을 문서화하지 않고 담당자 한 사람의 판단에 의존하면, 그 담당자가 조직을 떠나는 순간 로직은 손댈 수 없는 블랙박스가 된다. 이후 작은 규칙 변경조차 처음부터 다시 분석해야 하는 상황에 놓인다.

<표 1-1> 도구부터 도입했을 때 반복적으로 관찰되는 실패 패턴

실패 패턴	겉으로 드러나는 증상	근본 원인
예외 처리 폭증	자동화 밖으로 밀려나는 케이스 누적	기존 재량 경로를 제거하지 않고 그대로 둠
병목 이동	전체 처리 시간이 줄지 않음	프로세스 전체가 아닌 한 구간만 자동화
유지보수 불가	담당자 이탈 후 로직 수정 불가	예외·배경 지식이 문서화되지 않음

자료: 저자 정리

핵심 논지: 결함까지 그대로 자동화된다

세 증상은 각기 다른 얼굴을 하고 있지만 뿌리는 하나다. **프로세스 자체의 결함을 진단하지 않은 채 그 위에 도구를 얹으면, 자동화는 비효율을 없애는 것이 아니라 비효율이 반복되는 속도만 높인다.** 불필요한 승인 단계는 여전히 불필요한 채로 더 빨리 돌고, 중복 입력은 여전히 중복인 채로 더 자주 발생한다. 도구 선택을 아무리 신중히 해도 이 구조적 문제는 해결되지 않는다.

이 지점에서 흔히 나오는 반응은 “더 정교한 도구로 교체하면 되지 않느냐”는 것이다. 그러나 문제의 위치를 옮기는 것과 문제를 없애는 것은 다르다. RPA를 워크플로우 엔진으로, 워크플로우 엔진을 AI 에이전트로 바꾸어도 그 위에 얹힌 프로세스가 여전히 불필요한 단계와 모호한 책임 소재를 안고 있다면, 새 도구는 같은 결함을 더 세련된 방식으로 반복할 뿐이다. 도구 교체 주기가 짧아질수록 조직은 매번 새로운 학습 비용만 지불하고, 정작 프로세스의 결함은 손대지 않은 채 다음 도구로 넘어가는 순환에 빠진다.

시사점

자동화 프로젝트가 반복적으로 좌초하는 지점은 대개 기술 역량이 아니라 순서다. “어떤 도구를 쓸 것인가”를 먼저 묻는 조직은 프로세스의 결함을 그대로 이식하고, “이 단계가 왜 존재하는가”를 먼저 묻는 조직은 자동화할 대상 자체를 줄인 뒤 남은 것만 도구에 맡긴다.

이 책이 제안하는 순서

이러한 실패를 피하려면 도구 선택보다 프로세스 재설계가 먼저여야 한다. 이 책은 다섯 단계를 순서대로 밟는다. 먼저 손대기 전에 현재 프로세스를 있는 그대로 그리는 **진단**(2장)에서 시작해, 손덜 곳을 판단하는 **재설계 원칙**(3장)으로 좁힌다.

재설계가 끝난 다음에야 어떤 유형의 작업에 어떤 기술이 맞는지를 가르는 **기술 선택**(4장)으로 넘어가고, 전사에 한 번에 적용하는 대신 검증 후 넓히는 **로드맵**(5장)을 거쳐, 자동화가 굴러가기 시작한 뒤에도 주인이 있어야 유지된다는 **거버넌스**(6장)로 마무리한다. 마지막 7장은 이 순서를 한 번의 프로젝트로 끝내지 않고 조직의 습관으로 남기는 문제를 다룬다.

5

도구보다 앞에 두는 단계 — 진단 · 재설계 원칙 · 기술 선택 · 로드맵 · 거버넌스

이 순서를 지키는 조직과 건너뛰는 조직의 차이는 도구의 성능이 아니라 재설계 여부에서 갈린다. 도구부터 들이는 조직은 몇 달 안에 눈에 보이는 변화를 만들어 낼 수 있지만, 그 변화가 예외 처리 목록과 병목 이동으로 상쇄되는 과정을 다시 겪는다. 반면 프로세스를 먼저 다시 그리는 조직은 초기 착수는 느리게 보여도, 자동화를 엮을 대상 자체가 줄어 있으므로 이후의 확산과 유지보수가 훨씬 가볍다.

다음 장부터는 이 순서의 첫 단계, 즉 손대기 전에 현재 프로세스를 있는 그대로 그리는 진단에서 시작한다.

02

현재 프로세스 진단

자동화 대상을 정하기 전에 현재 프로세스를 부서·역할 단위로 있는 그대로 그리는 방법
과, 그 과정에서 드러나는 숨은 핸드오프·중복·대기 구간을 다룬다.

있는 그대로 그리는 법
매핑에서 드러나는 세 가지 낭비

자동화 대상을 고르는 회의는 대체로 담당자 인터뷰에서 시작한다. “이 업무는 이렇게 처리됩니다”라는 설명을 듣고, 그 설명을 순서도로 옮기고, 순서도 위의 반복 구간에 자동화 딱지를 붙인다. 문제는 그 설명이 실제 프로세스가 아니라 공식 절차라는 데 있다. 담당자는 매뉴얼에 적힌 순서를 말하지만, 실제 업무는 매뉴얼이 다루지 않는 예외와 우회로 채워져 있다.

마감이 급할 때 건너뛰는 확인 단계, 시스템이 처리하지 못해 메신저로 대신 넘기는 요청, 특정 상급자가 자리를 비웠을 때만 작동하는 대리 승인 — 이런 경로는 인터뷰에서 잘 언급되지 않는다. 담당자 본인에게는 너무 당연해서 ‘절차’로 인식되지 않기 때문이다. 그 결과 공식 절차만 자동화하면, 실제 업무를 지탱하던 우회로는 그대로 수작업으로 남고 자동화 시스템과 병행 운영되는 그림자 프로세스가 된다.

있는 그대로 그리는 법

이 괴리를 줄이는 방법은 질문의 방향을 바꾸는 데 있다. “어떻게 처리해야 합니까”가 아니라 “지난주에 실제로 무엇을 했습니까”를 묻고, 가능하면 화면 녹화나 처리 이력 로그처럼 말이 아닌 기록으로 확인한다. 담당자 한 명의 진술에 의존하지 않고 같은 업무를 다루는 여러 담당자에게 같은 질문을 던져 보는 것도 도움이 된다. 서로 다른 답이 나오는 지점이야말로 매뉴얼이 규정하지 못한, 그래서 사람마다 임의로 메워온 빈틈이기 때문이다.

이렇게 모은 단계를 부서·역할 단위의 레인으로 나눠 그리면, 각 단계가 어느 손에서 다음 손으로 넘어가는지가 드러난다. 신청자가 접수한 건이 1차 검토자를 거쳐 승인권자에게, 다시 시스템 담당자에게 넘어가는 흐름을 레인별로 배치하면 핸드오프가 몇 번 발생하는지, 어느 지점에서 되돌아가는 재작업이 생기는지가 한눈에 보인다. 이 시각화 자체가 진단의 결과물이다 — 아직 무엇을 자동화할지 정하기 전 단계다.

기록을 확인할 때는 시간을 짧게 잡더라도 처리의 처음부터 끝까지를 통째로 따라가는 편이 낫다. 접수부터 완료까지 한 건이 실제로 거친 화면과 담당자를 순서대로 추적하면, 설명에서 빠졌던 중간 단계가 자연스럽게 드러나기 때문이다.

이때 발견되는 우회로를 잘못된 관행으로 단정하고 기록에서 제외하면 진단은 다시 공식 절차의 복사본으로 돌아간다. 우회로는 대부분 현행 절차가 감당하지 못하는 상황을 현장이 스스로 메워 온 흔적이므로, 판단을 붙이지 않고 있는 그대로 적어 두는 것이 원칙이다. 어느 경로가 남을 만하고 어느 경로가 없어져야 하는지는 다음 장의 재설계 단계에서 가려도 늦지 않다.

3

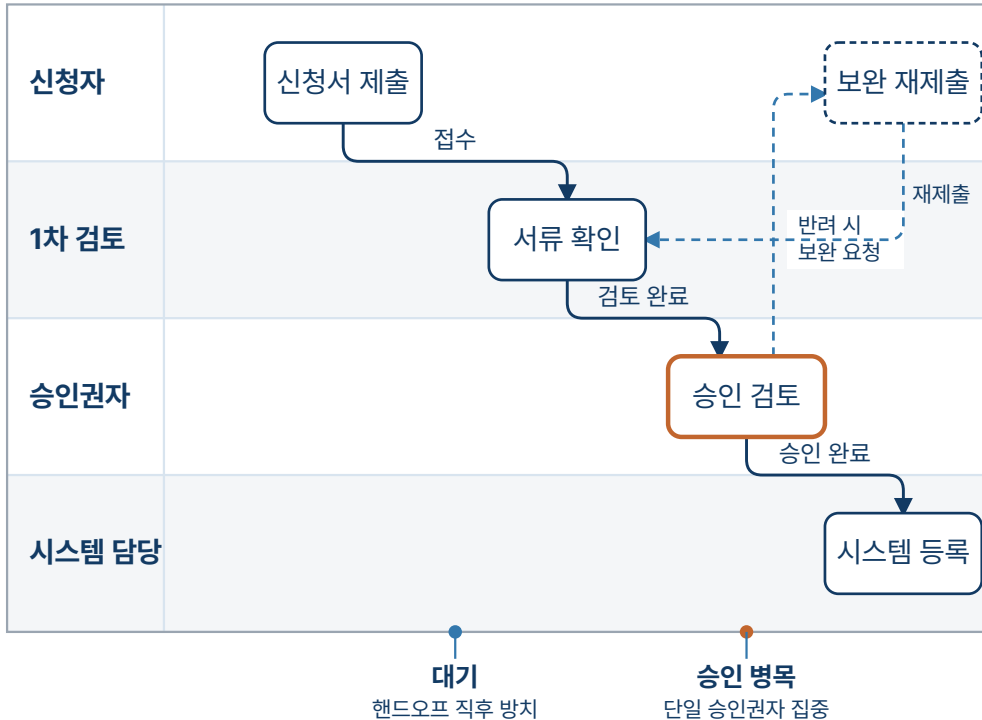
매핑이 드러내는 낭비 유형 — 대기 · 중복 입력 · 승인 병목

같은 이유로, 기록 확인은 담당자를 평가하는 자리가 아니라는 점을 처음부터 분명히 해 두어야 한다. 확인 결과가 인사 평가로 이어진다는 인상을 주면 담당자는 매뉴얼대로 처리한 건만 골라 보여주게 되고, 진단에서 가장 중요한 예외 경로는 다시 수면 아래로 숨는다. 진단 단계에서 확보해야 하는 것은 완성도 높은 도면이 아니라, 관련자들이 같은 그림을 보고 이야기할 수 있는 공통의 기록이다.

진단의 순서

매핑은 “무엇을 자동화할까”가 아니라 “실제로 무슨 일이 일어나는가”를 묻는 작업이다. 대상 선정은 그다음이다.

[그림 2-1] 신청서 처리 프로세스를 신청자·1차 검토·승인권자·시스템 담당 네 레인으로 나눠 핸드오프와 병목 지점을 표시한 스위밍레인



자료: 본문 정리

매핑에서 드러나는 세 가지 낭비

레인을 나눠 그려보면 거의 매 조직에서 반복적으로 관찰되는 낭비 유형이 있다. 첫째는 대기다. 한 레인에서 처리를 마친 건이 다음 레인 담당자의 확인을 받기까지 방치되는 구간으로, 담당자가 바빠서가 아니라 ‘넘어왔다는 사실’을 알리는 절차가 없어서 생기는 경우가 많다. 둘째는 중복 입력이다. 동일한 신청 정보를 접수 시스템, 검토용 스프레드시트, 승인 이메일에 각각 다시 옮겨 적는 구간으로, 시스템 간 연동이 없어 사람이 수작업으로 데이터를 이어 붙이는 지점에서 반복적으로 나타난다.

셋째는 승인 병목이다. 특정 승인권자 한 명에게 여러 갈래의 요청이 몰리면서, 그 사람의 일정이 곧 전체 프로세스의 처리 속도를 결정해버리는 구간이다. 세 유형 모두 개별 담당자의 역량

문제가 아니라 프로세스 구조의 문제라는 공통점이 있다 — 그래서 사람을 바꾸거나 다그쳐서는 해소되지 않는다.

낭비를 기록할 때도 세 유형의 이름을 그대로 붙여 두는 편이 좋다. 유형이 표시된 지도는 같은 문제가 여러 레인에서 동시에 반복되고 있다는 사실을 드러내고, 어느 유형부터 손대는 것이 파급이 큰지를 논의할 공통 근거가 된다. 이 우선순위 논의 역시 개인이 아니라 구조를 향해야 한다는 원칙은 그대로다.

<표 2-1> as-is 매핑에서 관찰되는 낭비 유형

낭비 유형	관찰되는 위치	표면적 원인으로 오해되기 쉬운 것
대기	레인 간 핸드오프 직후	담당자 태만
중복 입력	시스템이 연동되지 않은 접점	담당자 부주의
승인 병목	단일 승인권자에게 몰리는 구간	승인권자의 업무 처리 속도

자료: 본문 정리

흔한 오판

낭비의 원인을 담당자 개인에게 돌리면 진단이 거기서 멈춘다. 레인 구조 자체를 봐야 다음 장의 재설계가 성립한다.

세 낭비를 구분해서 기록해두는 이유는 다음 장의 재설계 원칙이 이 구분을 그대로 이어받기 때문이다. 대기는 핸드오프 방식의 문제이므로 통합으로, 중복 입력은 시스템 단절의 문제이므로 표준화로, 승인 병목은 권한 배분의 문제이므로 제거 또는 자동화로 접근하는 식이다. 진단 단계에서 이 세 유형을 뒤섞어 “여기저기 비효율이 많다”는 인상으로만 남기면, 재설계 단계에서 무엇부터 손대야 할지 다시 처음부터 찾아야 한다. 반대로 레인별로, 낭비 유형별로 구분해 그려두면 다음 단계는 이 그림 위에 원칙을 엮는 작업으로 바로 넘어갈 수 있다.

매핑 작업은 한 번의 워크숍으로 끝나지 않는 경우가 많다. 초안을 그려 관련 담당자들에게 돌려보면, 처음에는 언급되지 않았던 예외 경로가 뒤늦게 추가되곤 한다. 이 되먹임 과정 자체를 낭비로 여기지 않는 편이 좋다. 오히려 초안이 틀렸다는 것을 확인하는 절차이며, 몇 차례의 수정을 거쳐야 비로소 현장의 실제 흐름에 가까운 그림이 완성된다.

이렇게 완성된 as-is 지도는 이후 모든 재설계 논의의 공통 기준선이 된다 — 이 기준선이 없으면 재설계 회의는 각자가 서로 다른 프로세스를 머릿속에 그린 채로 진행된다. 기준선이 자리 잡으면 이후 장에서 다루는 재설계·기술 선택·확산의 모든 논의가 이 지도 위의 같은 지점을 가리키며 진행될 수 있다. 다음 장은 그 원칙을 제거·통합·표준화·자동화라는 네 개의 관문으로 구체화한다.

03

재설계 4원칙

프로세스를 자동화하기 전에 거쳐야 할 제거·통합·표준화·자동화의 순서와, 이 순서를 건너뛰면 왜 비싼 자동화가 반복되는지를 다룬다.

비효율을 자동화하면 비효율의 속도만 빨라진다
제거 — 이 단계는 왜 존재하는가
통합 — 통합 가능한 승인은 없는가
표준화 — 예외를 표준 경로로 흡수할 수 있는가
자동화 — 남은 반복 작업만 자동화 대상인가
순서를 건너뛰지 않는 것이 재설계의 전부다

자동화 프로젝트가 예산과 일정을 확보한 뒤 가장 먼저 하는 일은 대체로 도구를 정하는 일이다. 그러나 이 장이 제안하는 순서는 정반대다. 자동화는 네 가지 판단 중 마지막 하나일 뿐이며, 그 앞에 제거·통합·표준화라는 세 개의 관문을 통과한 작업만 자동화 대상이 될 자격을 얻는다. 이 순서를 어떤 이름으로 부르든 핵심은 순서 그 자체에 있다 — 자동화를 늦게 물을수록 자동화의 효과는 오래간다.

이 장은 그 네 관문을 순서대로 살펴보고, 각 관문에서 무엇을 물어야 하는지를 판단 질문의 형태로 정리한다. 관문마다 판단의 주체와 근거가 다르므로, 순서를 지키는 것 자체가 판단의 품질을 지키는 장치가 된다. 네 관문을 통과한 결과는 장 끝의 판단 흐름도로 요약된다.

비효율을 자동화하면 비효율의 속도만 빨라진다

앞 장에서 확인한 대기·중복 입력·승인 병목은 프로세스에 원래 있던 결함이다. 이 결함을 그대로 둔 채 RPA 봇이나 워크플로우 엔진을 얹으면 결함이 사라지는 것이 아니라 결함을 통과하는 속도만 빨라진다. 필요 없는 승인 단계는 여전히 필요 없는 채로 자동 상신되고, 중복 입력은 여전히 두 번 입력되지만 더 빠르게 두 번 입력된다. 자동화 예산을 결함 위에 쓰는 셈이다.

그래서 재설계는 ‘무엇을 자동화할 것인가’를 묻기 전에 ‘무엇을 자동화 대상에서 아예 제외할 것인가’를 먼저 물어야 한다. 이 단계에서 걸러내야 하는 것은 눈에 보이는 비효율만이 아니다. 앞 장의 as-is 지도에서 확인된 대기·중복 입력·승인 병목 각각에 대해 그 구간이 애초에 왜 생겼는지를 먼저 묻지 않으면, 자동화는 그 구간의 존재 자체를 승인해 버리는 결정이 된다. 결함 위에 속도를 얹는 대신 결함부터 걷어내는 것 — 재설계 4원칙이 제거에서 시작하는 이유다.

이 판단은 담당 부서 혼자 내릴 수 있는 것이 아니어서, 재설계 초기에 관련 부서가 같은 지도를 놓고 합의하는 자리가 필요하다. 합의 없이 걷어낸 결함은 얼마 지나지 않아 다른 이름으로 되 돌아온다.

제거 — 이 단계는 왜 존재하는가

첫 관문은 제거다. 프로세스의 각 단계 앞에서 던질 질문은 하나, “이 단계는 왜 존재하는가”다. 특정 사고 이후 임시로 추가된 확인란, 조직 개편 전 담당자 기준으로 설계된 결재선, 상위 시스템이 이미 검증한 항목을 하위 단계에서 다시 검증하는 절차는 대체로 최초 도입 이유가 사라진 뒤에도 관성으로 남아 있다. 이런 단계는 자동화가 아니라 삭제의 대상이다. 삭제로 없어지는 단계는 자동화 비용도 함께 없앤다.

제거를 판단할 권한이 실무 담당자에게 없는 경우가 많다는 점도 이 단계의 특징이다. 확인란 하나를 없애는 데도 그 확인란을 만든 부서의 동의가 필요하므로, 제거 관문은 실무 조사와 별도로 결정 권한을 가진 사람이 참여할 때만 실제로 닫힌다. 참여자가 없으면 ‘없애자’는 결론은 회의록에만 남고 프로세스는 그대로 남는다. 그래서 제거 관문의 산출물은 ‘없앨 단계 목록’이 아니라 ‘없애기로 결정된 단계 목록’이어야 하고, 항목마다 결정한 사람과 근거가 함께 남아야 한다.

제거 관문의 질문

이 단계는 왜 존재하는가 — 답이 '원래 그랬다'뿐이라면 그 단계는 자동화가 아니라 삭제의 후보다.

통합 — 통합 가능한 승인은 없는가

제거하고 남은 단계에는 통합을 묻는다. 서로 다른 시점에 서로 다른 담당자가 검토하는 승인 두세 개가 사실은 같은 기준으로 판단하고 있지 않은지 확인하는 작업이다. 금액 기준 결재와 예산 항목 확인이 별도 단계로 나뉘어 있지만 실제로는 같은 자료를 보고 같은 결재선이 처리한다면, 두 단계를 하나의 검토 지점으로 합칠 수 있다. 통합은 처리 건수를 줄이지 않지만 핸드오프 횟수를 줄이며, 핸드오프가 줄어든 만큼 대기 시간도 함께 줄어든다.

통합 후보를 찾을 때는 승인 서류의 첨부 자료를 비교해 보는 것이 지름길이다. 서로 다른 승인 단계가 같은 자료를 요구하고 있다면, 두 단계는 이미 같은 판단을 두 번 하고 있을 가능성이 높다.

통합을 반대하는 목소리는 대개 '우리 부서의 검토 관점은 다르다'는 형태로 나타난다. 이때 관점이 실제로 다른지 확인하는 방법 역시 기록이다. 최근 처리된 승인 건에서 두 단계가 서로 다른 결론을 낸 사례가 얼마나 있었는지를 살펴보면, 관점의 차이가 실재하는지 관행의 관성인지가 드러난다. 결론이 같린 사례가 드물다면 두 검토는 사실상 하나의 판단이며, 통합의 근거는 이미 기록 안에 있는 셈이다. 기록이 말해 주는 것을 그대로 따르는 것이 통합 논의를 감정 싸움으로 만들지 않는 가장 빠른 길이기도 하다.

표준화 — 예외를 표준 경로로 흡수할 수 있는가

세 번째 관문은 표준화다. 같은 유형의 요청이 부서마다, 담당자마다 다른 서식과 다른 절차로 처리된다면 그 다양성 자체가 자동화를 가로막는 벽이 된다. 자동화 로직은 입력과 처리 경로가 일정할 때만 안정적으로 작동하기 때문이다.

예를 들어 같은 성격의 구매 요청이 부서별로 서로 다른 양식과 서로 다른 결재 순서를 거친다면, 자동화는 부서 수만큼의 예외 분기를 떠안게 된다. 표준화 단계에서는 지금 예외로 취급되는 처리 방식 중 얼마나 많은 부분이 사실은 표준 경로로 흡수될 수 있는지를 검토한다. 법적 사유나 특수 계약처럼 성격이 다른 진짜 예외만 남기고 나머지를 하나의 경로로 모으면, 그 경로만 자동화해도 전체 처리량의 대부분을 감당할 수 있다.

표준 경로가 하나로 정리되면 예외 처리의 기준도 '표준에서 얼마나 벗어났는가'라는 하나의 잣대로 통일된다. 표준화가 어려운 이유는 기술이 아니라 합의에 있다.

부서마다 자기 양식이 익숙하고, 양식을 바꾸는 쪽이 불편을 떠안기 때문이다. 그래서 표준 경로를 정할 때는 가장 처리량이 많은 경로를 기준으로 삼고, 나머지 부서가 옮겨 오는 비용을 재설계 일정에 명시적으로 포함하는 편이 현실적이다. 합의 비용을 치르지 않은 표준은 문서로만 남고, 현장은 각자의 예외로 되돌아간다. 반대로 옮겨 오는 비용이 일정에 반영된 표준은 각 부서가 언제까지 무엇을 바꾸면 되는지가 분명해, 합의가 실행으로 이어질 가능성이 훨씬 높다.

자동화 — 남은 반복 작업만 자동화 대상인가

앞의 세 관문을 통과하고 남은 것이 비로소 자동화 대상이다. 이 시점의 자동화 대상은 이미 불필요한 단계가 제거되고 승인이 통합되고 경로가 표준화된 상태이므로 자동화 로직 자체가 단순해진다.

반대로 순서를 건너뛰고 곧바로 자동화에 들어간 조직은 예외 분기가 계속 늘어나는 로직을 유지보수해야 하는 상황에 놓이기 쉽다. 담당자 한 사람만 그 예외 목록을 기억하고 있다면, 그 담당자가 자리를 옮기는 순간 자동화는 손댈 수 없는 블랙박스로 남는다. 자동화는 남은 반복 작업에만, 그것도 마지막 순서로 적용해야 효과가 오래간다. 이렇게 좁혀진 자동화 대상 목록은 다음 장에서 기술을 고를 때 그대로 입력이 되므로, 관문의 기록을 남겨 두는 일 자체가 다음 단계의 준비다.

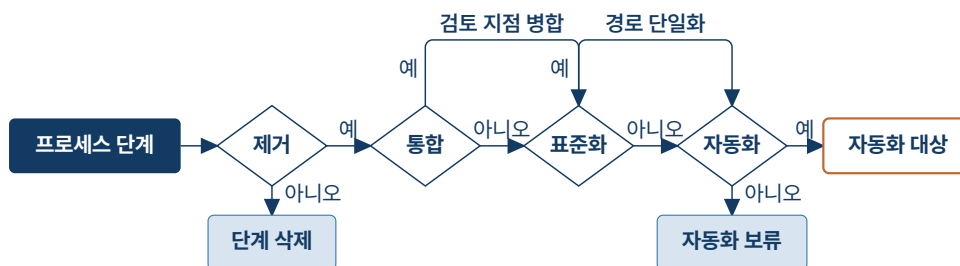
<표 3-1> 재설계 4원칙과 단계별 판단 질문

원칙	판단 질문	통과하지 못하면
제거	이 단계는 왜 존재하는가	단계 삭제
통합	통합 가능한 승인은 없는가	검토 지점 병합
표준화	예외를 표준 경로로 흡수할 수 있는가	경로 단일화
자동화	반복 작업만 남았는가	자동화 보류

자료: 저자 정리

이 네 질문은 한 번에 끝나는 체크리스트가 아니라 프로세스의 모든 단계마다 순서대로 되풀이해 던져야 하는 판단 로직이다. [그림 3-1]은 이 판단이 단계마다 어떻게 이어지는지를 보여준다. 어느 관문에서든 답이 '아니오'로 갈리는 순간 프로세스는 왼쪽 경로 — 삭제·병합·단일화 — 로 빠지고, 네 관문을 모두 통과한 단계만 오른쪽 끝의 자동화 대상에 도달한다.

[그림 3-1] 프로세스 단계마다 제거·통합·표준화·자동화 여부를 순서대로 판단하는 흐름



자료: 저자 작성, 재설계 4원칙 재구성

순서를 건너뛰지 않는 것이 재설계의 전부다

재설계 4원칙을 적용할 때

네 단계를 한 프로세스에 동시에 적용하려 하지 말고 단계별로 끝을 낸 뒤 다음 단계로 넘어가라. 제거할 것을 다 제거하기 전에 통합을 논의하면, 원래 없어졌어야 할 단계를 통합안에 포함시키는 실수가 반복된다.

재설계 4원칙의 목적은 자동화를 늦추는 것이 아니라 자동화가 거닐 표적을 정확하게 좁히는 데 있다. 제거·통합·표준화를 거친 프로세스는 자동화 로직이 단순해지고, 단순한 로직은 담당자가 바뀌어도 유지보수가 가능하다. 이 순서를 지키는 조직과 건너뛰는 조직의 차이는 어떤 자동화 기술을 고르느냐보다 먼저, 무엇을 자동화 대상으로 남기느냐에서 갈린다. 그 차이를 만드는 것은 새 도구가 아니라, 모든 단계 앞에서 같은 네 질문을 던지는 규율이다. 규율은 화려하지 않지만, 다음 장에서 다룰 기술 선택이 헛돌지 않게 하는 유일한 밑돌이다.

4

자동화 앞의 관문 — 제거 · 통합 · 표준화
· 자동화

04

자동화 기술 지형도

RPA·워크플로우 엔진·API 연동·AI 에이전트가 각각 어떤 유형의 반복 작업에 맞는지, 그리고 이들이 하나의 처리 흐름 안에서 어떻게 역할을 나누는지를 다룬다.

데이터 레이어를 중심으로 만나는 네 기술
화면을 대신 조작하는 기술, RPA
흐름을 관리하는 워크플로우 엔진
시스템과 시스템을 잇는 API 연동
판단이 필요한 구간의 AI 에이전트
하나의 요청이 지나가는 다섯 상태

앞 장의 제거·통합·표준화 단계를 거치고 나면, 자동화 대상은 처음보다 훨씬 작아져 있다. 남는 것은 없앨 수도 합칠 수도 없는 반복 작업뿐이다. 문제는 여기서부터다. 남은 반복 작업이 모두 같은 성격은 아니기 때문에, 하나의 기술로 전부를 처리하려 하면 또 다른 무리가 생긴다. 화면을 옮겨 다니는 단순 반복, 여러 담당자를 거쳐야 하는 승인 흐름, 시스템끼리 주고받아야 하는 데이터, 그리고 규칙만으로는 판단이 서지 않는 예외까지 — 성격이 다른 네 유형의 작업에는 성격이 다른 네 기술이 대응한다.

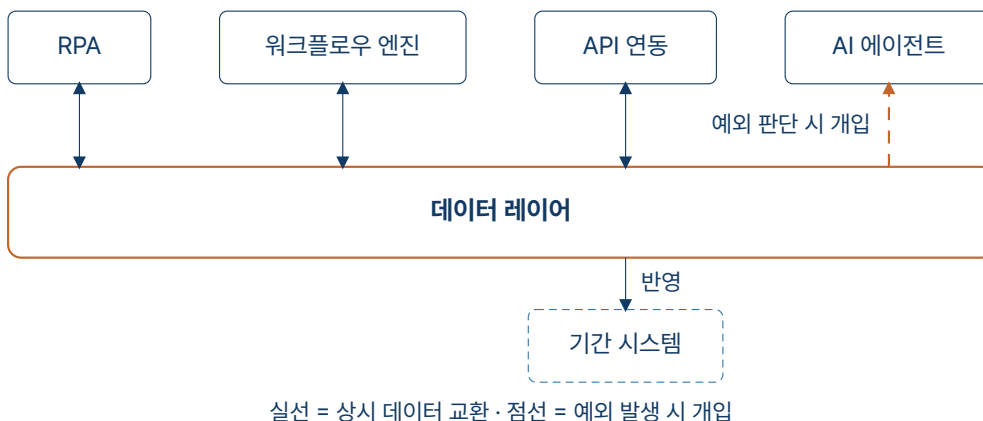
이 장에서는 이 네 기술을 순서대로 살펴보고, 각 기술이 몇 초 빠르거나 얼마나 정확한지를 비교하지 않는다. 그런 수치는 도구·환경마다 달라 일반화하기 어렵고, 무엇보다 기술 비교표만으로는 어느 구간에 무엇을 놓아야 하는지가 정해지지 않는다. 대신 앞 장에서 남은 각 작업 유형의 성격과, 그 성격이 어떤 기술의 작동 방식과 맞물리는지를 짚지어 본다. 기술 이름을 먼저 외우고 적용처를 찾는 순서를 뒤집어, 작업의 성격에서 기술로 거슬러 올라가는 순서다. 화면, 흐름, 데이터, 판단 — 사람이 하던 일의 겉에서 속으로 들어가는 순서이기도 하다.

데이터 레이어를 중심으로 만나는 네 기술

RPA·워크플로우 엔진·API 연동·AI 에이전트는 서로 무관하게 따로 도입되는 경우가 많지만, 실제 처리 흐름에서는 같은 데이터 레이어를 사이에 두고 맞물려 움직인다. RPA가 화면에서 읽어 온 값이 워크플로우 엔진의 승인 단계로 넘어가고, 워크플로우 엔진의 결정이 API 연동을 통해 기간 시스템에 반영되며, 규칙으로 판단할 수 없는 구간에서는 AI 에이전트가 끼어들어 나머지 흐름을 이어받는다. 이 연결 구조를 먼저 그려 두어야 이후 각 기술의 역할이 어디서 시작해 어디서 끝나는지가 분명해진다.

네 기술을 각각 도입하더라도 데이터 레이어와의 접점을 먼저 정의해 두면 기술 하나를 교체할 때도 나머지 연결은 그대로 유지되고, 점점 정의가 없으면 기술들은 서로의 화면과 파일을 직접 읽는 임시 연결로 얹히기 시작한다. 접점을 그림 한 장으로 유지하는 비용은 작지만, 그 그림이 없을 때 치르는 통합 비용은 기술 수에 비례해 커진다.

[그림 4-1] RPA·워크플로우 엔진·API 연동·AI 에이전트가 데이터 레이어를 중심으로 연결 되는 구조



자료: 본문 정리

화면을 대신 조작하는 기술, RPA

RPA는 사람이 화면을 보고 클릭·입력하던 동작을 그대로 흉내 내는 기술이다. 시스템 간 연동이 되어 있지 않아 담당자가 한 화면의 값을 읽어 다른 화면에 옮겨 적던 구간, 즉 3장에서 표준화까지는 마쳤지만 시스템 자체를 바꿀 수 없어 수작업이 남아 있는 구간에 정확히 맞는다. 반대로 화면 구조가 자주 바뀌거나 판단이 섞인 작업에 RPA를 얹으면, 화면이 바뀔 때마다 자동화가 함께 멈추는 유지보수 부담으로 되돌아온다.

그래서 RPA를 배치할 때는 대상 화면이 얼마나 자주 바뀌는지를 먼저 확인하는 것이 순서다. 조직이 직접 통제하지 못하는 외부 사이트나 개편 주기가 잦은 시스템의 화면은 RPA가 기대만큼 안정적으로 버티기 어렵고, 반대로 오래 유지되어 온 내부 기간 시스템의 화면은 상대적으로 안전한 표적이 된다. RPA는 시스템을 바꾸지 않고 손을 대신하는 기술이므로, 화면이라는 접점이 안정적일수록 수명이 길어진다.

화면 안정성을 확인하는 가장 간단한 방법은 최근 몇 번의 시스템 개편에서 해당 화면이 바뀌었는지를 담당 부서에 묻는 것이다. 개편 이력이 잦은 화면이라면 RPA 대신 API 연동의 가능성을 먼저 타진하는 편이 안전하고, 곧 시스템 교체가 예정된 화면도 후순위다. 자동화의 수명이 화면의 수명을 넘을 수 없기 때문이다. 이 확인은 자동화 담당 혼자서는 어렵고, 화면을 소유한 시스템 부서의 개편 계획을 미리 공유받는 창구가 있어야 가능하다.

4

성격이 다른 작업 유형 — 화면 조작 · 흐름 · 데이터 전달 · 판단

흐름을 관리하는 워크플로우 엔진

워크플로우 엔진은 하나의 요청이 여러 담당자와 부서를 거치는 순서, 즉 2장에서 스위밍레인으로 그렸던 핸드오프 구조 자체를 시스템으로 관리한다. 누구에게 넘어가야 하는지, 얼마나 기다리면 재촉 알림을 보낼지, 승인이 거부되면 어디로 되돌아가는지를 규칙으로 정의해 둔다. RPA가 개별 동작을 대신한다면, 워크플로우 엔진은 그 동작들이 이어지는 순서와 책임의 이동을 대신한다.

워크플로우 엔진의 또 다른 효용은 흐름이 기록으로 남는다는 데 있다. 어떤 요청이 어느 단계에서 얼마나 머물렀는지가 시스템에 쌓이므로, 2장에서 손으로 그렸던 as-is 지도를 이후에는 실측 기록으로 대신할 수 있게 된다. 병목이 어디로 옮겨 갔는지, 재촉 알림이 어느 구간에서 반복되는지가 데이터로 드러나면, 다음 재설계의 진단 비용은 그만큼 줄어든다.

기록이 남는다는 것은 되돌아볼 수 있다는 뜻이기도 하다. 전환 이후 흐름이 기대대로 움직이는지를 5장의 병행 운영 단계에서 검증할 때도 이 기록이 비교의 근거가 된다. 다만 엔진이 관리하는 것은 흐름의 뼈대이므로, 개별 단계 안의 세부 동작은 여전히 RPA나 API 연동의 몫으로 남는다. 그 경계를 흐리면 엔진 규칙 안에 화면 조작 스크립트가 섞여 들어가 유지보수 지점이 다시 흩어진다.

흐름 관리의 본질

워크플로우 엔진이 대신하는 것은 개별 동작이 아니라 동작들이 이어지는 순서와 책임의 이동이다. 사람이 하던 '넘기고 기다리고 재촉하는' 일이 규칙으로 옮겨 간다.

시스템과 시스템을 잇는 API 연동

API 연동은 사람의 화면 조작을 거치지 않고 시스템끼리 데이터를 직접 주고받는 방식이다. 화면을 흉내 내는 RPA와 달리 두 시스템 사이의 정해진 인터페이스를 통해 값을 전달하므로, 처리 방식이 일정하고 화면 변경의 영향을 받지 않는다.

다만 상대 시스템이 그런 인터페이스를 열어 두지 않았거나 조직 내부에서 열 권한이 없는 경우가 많아, API 연동이 가능한 구간과 RPA로 남겨 둘 수밖에 없는 구간이 실무에서는 함께 존재한다. 그래서 두 기술은 서로 대체재라기보다, 같은 시스템 간 연결을 두고 인터페이스가 열려 있는지 여부로 갈리는 이웃 관계에 가깝다.

인터페이스가 새로 열리면 그만큼 RPA로 남아 있던 구간이 API 연동으로 옮겨 가고, 그 반대는 거의 일어나지 않는다. 그래서 연동 구간을 설계할 때는 지금 열려 있는 인터페이스만이 아니라 앞으로 열릴 가능성이 있는 인터페이스까지 함께 확인해 두는 편이 좋다. 권한 협의에 걸리는 시간은 개발 기간보다 길어지기 일쑤여서, 인터페이스 확인은 기술 선택과 동시에 시작해야 일정이 맞는다.

협의 상대가 조직 밖이라면 계약과 보안 검토까지 없으므로, 이 구간의 일정은 넉넉하게 잡는 편이 안전하다. 이런 이유로 API 연동은 기술 검토보다 관계 부서와의 조율이 프로젝트의 실질적인 경로가 되는 경우가 많다.

기술 선택의 기준

어떤 기술을 쓸지는 성능이 아니라 작업의 성격이 정한다. 반복되는 화면 조작인가, 담당자 사이의 흐름인가, 시스템 간 데이터 전달인가, 아니면 정해진 규칙만으로는 답이 갈리는 판단인가 — 이 구분이 먼저이고 도구는 그다음이다.

판단이 필요한 구간의 AI 에이전트

앞의 세 기술은 모두 정해진 규칙을 정해진 순서대로 수행한다는 공통점이 있다. 그러나 예외 처리처럼 입력마다 형태가 다르고 규칙으로 미리 다 나열할 수 없는 구간에서는 규칙 기반 자동화가 오히려 예외 목록만 계속 늘리는 결과를 낳는다. AI 에이전트는 이 비정형 판단 구간, 즉 문서를 읽고 상황을 분류하거나 애매한 요청의 의도를 파악해 다음 단계를 고르는 지점에 배치된다.

앞 세 기술이 처리한 정형 흐름 위에, 판단이 필요한 마디에서만 개입하는 방식이 안정적이다. 반대로 정형 처리까지 통째로 AI 에이전트에 맡기면, 규칙으로 충분히 보장되던 구간의 결과까지

지 매번 판단에 따라 달라질 수 있는 구간으로 바뀐다. 판단이 필요한 마디와 규칙으로 충분한 마디를 가르는 일은, 결국 3장에서 표준화 관문이 남겨 둔 '진짜 예외'의 목록을 다시 확인하는 일이기도 하다.

판단 구간을 좁게 유지할수록 검증도 쉬워진다. 규칙 구간은 결과가 정해져 있어 자동 검증이 가능하지만, 판단 구간은 사람이 표본을 확인하는 절차가 필요하기 때문이다. 표본 확인의 부담 역시 판단 구간의 크기에 비례하므로, 판단을 좁힌 설계는 운영 비용까지 함께 줄이는 설계가 된다.

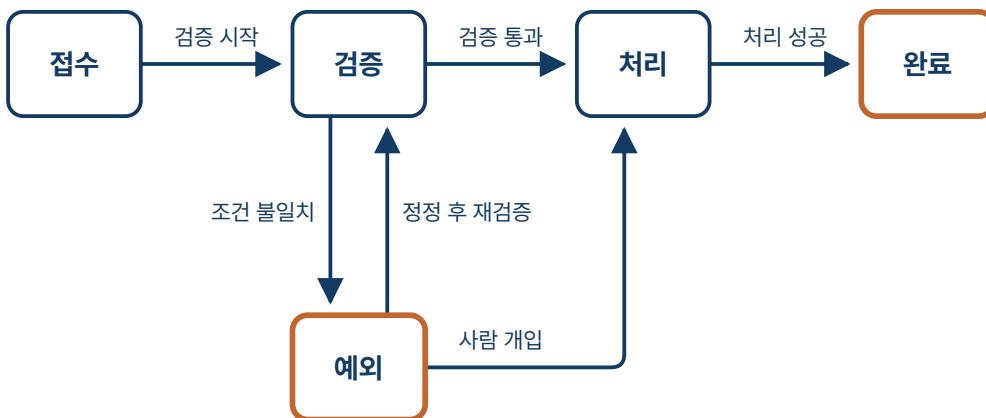
판단 결과를 기록으로 남겨 두면, 반복되는 판단 유형이 규칙으로 승격될 후보라는 사실도 드러난다. 판단 구간이 규칙으로 옮겨 갈수록 흐름 전체의 예측 가능성은 그만큼 높아진다.

하나의 요청이 지나가는 다섯 상태

기술 배치가 끝나면, 하나의 요청이 실제로 이 흐름을 통과하며 겪는 상태를 살펴볼 차례다. 요청은 접수 상태로 시작해 검증을 거치고, 검증을 통과하면 처리 상태로 넘어간다. 검증에서 형식이나 조건이 맞지 않으면 예외 상태로 빠지며, 예외 상태는 정정 후 재검증으로 돌아가거나 사람의 개입으로 이어진다. 처리가 정상적으로 끝나면 완료 상태에 도달한다.

이 다섯 상태 사이의 전이 조건을 명시적으로 정의해 두어야, 어느 구간에 RPA·워크플로우 엔진·API 연동·AI 에이전트 중 무엇을 배치할지가 상태 단위로 분명해진다. 상태의 이름과 전이 조건은 조직마다 달라질 수 있지만, 어느 상태에서 어느 상태로 넘어갈 수 있는가를 명시한다는 원칙은 같다.

[그림 4-2] 요청이 접수·검증·처리·예외·완료 상태를 거치며 전이되는 조건



자료: 본문 정리

시사점

상태머신으로 흐름을 정의해 두면 새 기술이 추가되거나 교체되어도 상태와 전이 조건은 그대로 남는다. 기술은 상태 사이를 채우는 실행 수단일 뿐, 흐름의 구조 자체는 아니다.

05

파일럿에서 확산까지

재설계한 프로세스를 전사에 한 번에 적용하지 않고, 단일 프로세스 파일럿에서 검증한 뒤 단계적으로 확산하는 실행 순서를 다룬다.

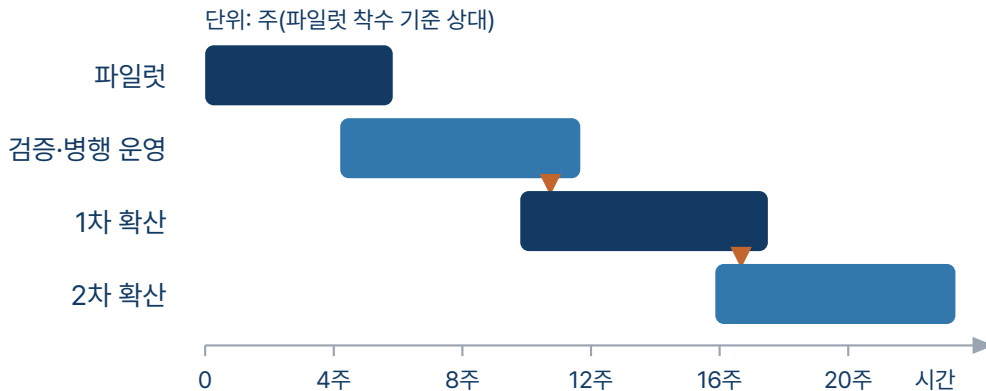
파일럿 대상을 고르는 기준

병행 운영 — 두 흐름을 동시에 돌린다

확산은 인접 프로세스부터

앞 장에서 고른 기술을 재설계된 프로세스에 얹었다고 해서 곧바로 전사에 풀어놓을 수는 없다. 여러 부서가 동시에 새 흐름으로 갈아타면 문제가 생겼을 때 원인을 어디서부터 짚어야 할지 알 수 없고, 되돌릴 지점도 애매해진다. 그래서 재설계는 언제나 하나의 프로세스에서 먼저 검증한 뒤, 검증된 것만 옆으로 넓히는 순서를 밟는다. 이 장은 그 순서 — 파일럿, 병행 운영, 확산 — 을 시간축 위에서 다룬다.

[그림 5-1] 파일럿·검증·1차 확산·2차 확산 구간이 시간축에서 겹치며 이어지는 순서



자료: 본문 절차 정리

[그림 5-1]처럼 네 구간은 순서대로 끝나고 시작하는 것이 아니라 일부 겹친다. 검증이 끝나기 전에 1차 확산 준비가 이미 시작되고, 1차 확산이 안정화되는 동안 2차 확산 대상이 정해진다. 이 겹침 구조를 먼저 눈에 담아 두면, 아래에서 각 구간이 왜 그렇게 배치되는지가 자연스럽게 이어진다.

파일럿 대상을 고르는 기준

파일럿으로 삼을 프로세스는 화려한 곳이 아니라 조용한 곳에서 골라야 한다. 기준은 두 가지다. 첫째, 발생 빈도가 높아야 한다. 자주 돌아가야 짧은 기간 안에도 충분한 실행 사례가 쌓이고, 새 흐름이 실제로 안정적인지 판단할 근거가 빨리 모인다. 둘째, 예외 경로가 적어야 한다. 예외가 많은 프로세스를 처음부터 파일럿으로 잡으면, 새 흐름의 결함인지 애초에 프로세스가 복잡한 탓인지 구분되지 않는다.

2

파일럿 기준 — 높은 빈도·적은 예외

파일럿 선정 체크

빈도는 높고 예외는 적은 프로세스를 고른다. 화제성이나 조직 내 상징성으로 대상을 정하면 검증이 아니라 시범 행사가 된다.

이 기준에 맞는 프로세스는 대개 눈에 띄지 않는 반복 업무 쪽에 있다. 3장에서 이미 제거·통합·표준화를 거쳤다면, 남은 반복 구간일수록 파일럿 후보로 적합하다. 반대로 발생 빈도는 낮지만 조직 내 상징성이 큰 프로세스(예: 임원 결재가 걸린 특수 승인)를 파일럿으로 고르는 경우가 있는데, 이런 선택은 검증 속도를 늦추는 동시에 예외 처리 설계에 자원을 과도하게 쓰게 만든다. 파일럿의 목적은 새 흐름을 자랑하는 것이 아니라 짧은 기간 안에 결함을 최대한 많이 드러내는 것이므로, 눈에 띄지 않아도 자주 반복되는 프로세스가 더 효율적인 선택지다.

병행 운영 — 두 흐름을 동시에 돌린다

파일럿 기간에는 기존 방식과 새 흐름을 동시에 돌린다. 새 흐름만 믿고 기존 방식을 바로 끄면, 새 흐름이 놓친 처리 건이 그대로 유실된다. 병행 운영은 이 위험을 없애는 동시에 비교 자료를 만든다. 같은 입력이 두 경로를 거친 결과를 나란히 놓고 처리 시간, 오류 발생 지점, 담당자의 개입 빈도를 견주면 새 흐름이 실제로 개선인지 판단할 수 있다.

병행 운영은 수 주 단위로 잡는 편이 현실적이다. 지나치게 짧으면 드물게만 나타나는 예외 케이스를 못 보고 넘어가고, 지나치게 길면 두 흐름을 동시에 관리하는 비용이 검증의 이점을 갉아먹는다. 이 기간 동안의 검증 기준도 미리 정해 두어야 한다 — 처리 시간이 늘었는지 줄었는지 감으로 판단하지 말고, 무엇을 비교하면 전환 여부를 결정할 수 있는지를 병행 운영에 들어가기 전에 합의해야 기간이 끝났을 때 판단이 흔들리지 않는다.

병행 운영을 건너뛸 때

새 흐름을 검증 없이 바로 전환하면 실패 원인을 프로세스 설계와 도구 오작동 중 무엇으로도 좁힐 수 없다. 되돌릴 기준선 자체가 사라지기 때문이다.

확산은 인접 프로세스부터

파일럿이 검증을 통과하면 다음은 확산이다. 이때도 순서가 있다. 가장 먼 부서나 가장 규모가 큰 영역이 아니라, 파일럿 프로세스와 입력·출력을 주고받는 인접 프로세스부터 넓힌다. 이유는 단순하다. 인접 프로세스는 이미 검증된 흐름과 데이터 형식·승인 구조를 상당 부분 공유하고 있어서, 파일럿에서 얻은 학습을 거의 그대로 옮길 수 있다. 반면 접점이 없는 먼 프로세스로 바로 건너뛰면 매번 처음부터 새로 검증하는 셈이 된다.

1차 확산이 안정 궤도에 오르면 그 경계에 닿아 있는 다음 프로세스로 2차 확산을 넓힌다. 이렇게 인접 구간을 따라 번져 나가는 방식은 속도는 느려 보여도, 각 단계에서 실패가 나더라도 그 파급이 이미 검증된 구간을 넘어가지 않는다는 이점이 있다. 문제가 생겼을 때 원인을 새로 확산된 구간 안으로 좁혀서 찾을 수 있고, 이미 안정화된 파일럿 구간까지 되돌아가 다시 점검할 필요가 없다.

전사 동시 적용이 매력적으로 보이는 것은 속도 때문이지만, 재설계된 프로세스가 실제로 버티는지는 결국 이렇게 한 구간씩 넓혀 가며 확인하는 수밖에 없다. 확산 대상을 정할 때마다 앞서 정한 병행 운영 기준을 같은 잣대로 다시 적용하는 것이 이 방식의 핵심이다. 기준을 구간마다 느슨하게 풀면 확산은 빨라지지만, 그만큼 거버넌스 부담이 뒤로 밀릴 뿐이다.

06

거버넌스와 오너십

자동화된 프로세스가 굴러가기 시작한 뒤 누가 예외를 처리하고 누가 변경을 승인할지를
정하지 않으면 왜 다시 수작업으로 돌아가는지를 다룬다.

담당자 한 명에게 남은 로직
예외는 어디로 가야 하는가
변경은 누가 승인하는가

자동화는 배포된 순간이 아니라 배포된 다음 날부터 시험대에 오른다. 재설계된 프로세스가 실제 운영에 들어가면 규정 변경, 예외 상황, 시스템 업데이트가 끊임없이 자동화 로직과 충돌한다. 이 장은 자동화가 커진 뒤에도 조직이 갖춰야 할 운영 체계 — 누가 예외를 받아 처리하고, 누가 변경을 승인하며, 누가 결과에 책임을 지는지를 다룬다.

담당자 한 명에게 남은 로직

많은 자동화 프로젝트는 구축 단계에 공을 들이고 운영 단계는 방치한다. 초기 구축을 이끈 담당자가 자동화 로직의 세부 조건을 머릿속에만 담아 두는 경우가 흔하다. 이 담당자가 이동하거나 퇴사하면, 남은 조직은 자동화가 왜 특정 조건에서 특정 방식으로 동작하는지 설명할 수 없는 상태에 놓인다.

문서화되지 않은 로직의 대가

자동화 규칙이 담당자의 암묵지로서만 존재하면, 그 담당자가 자리를 비우는 순간 유지보수는 사실상 중단된다. 오류가 발생해도 원인을 추적할 사람이 없어 수작업으로 되돌아가는 경우가 관찰된다.

이 문제의 해법은 기술적이지 않다. 자동화 로직을 문서로 남기고 갱신 이력을 관리하는 습관은 조직 역량이지 도구의 기능이 아니다. 로직을 소유한 담당자가 바뀌어도 다음 담당자가 같은 판단 근거를 이어받을 수 있어야 재설계의 효과가 유지된다.

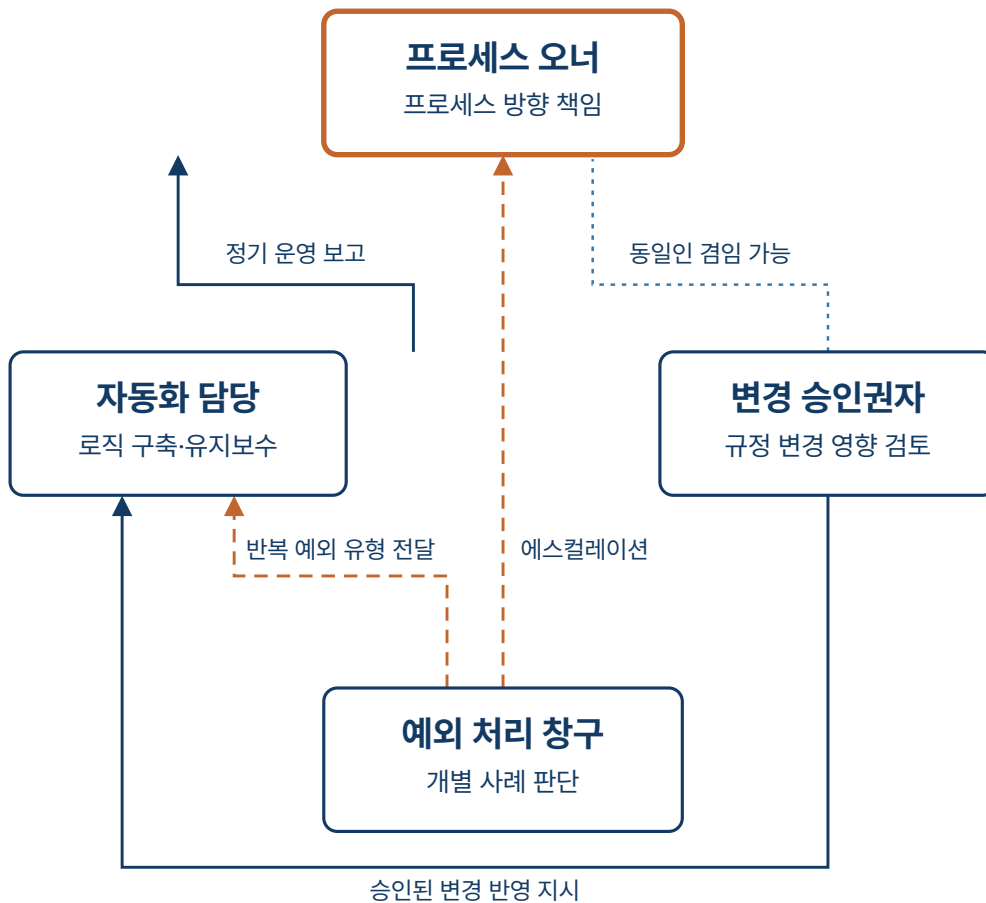
예외는 어디로 가야 하는가

아무리 정교하게 설계해도 자동화가 처리하지 못하는 사례는 남는다. 입력값이 정의된 범위를 벗어나거나, 예상치 못한 조합의 조건이 들어오거나, 상위 규정이 갱신되기 전 과도기의 요청이 그렇다. 문제는 예외가 생긴다는 사실이 아니라, 예외가 생겼을 때 누구에게 가야 하는지가 정해져 있지 않다는 사실이다.

경로가 없는 예외는 두 가지 방식으로 조직을 갇아먹는다. 하나는 담당자마다 임의로 판단해 처리 결과가 들쭉날쭉해지는 것이고, 다른 하나는 아무도 책임지지 않으려 해 요청이 대기 상태로 방치되는 것이다. 두 경우 모두 자동화 이전보다 처리가 느려지는 역설이 발생한다. 들쭉날쭉한 판단은 시간이 지나면 더 나쁜 형태로 굳는다. 담당자마다 다른 판단이 각자의 선례로 쌓이면, 같은 요청이 창구에 따라 다른 결론을 받는 상태가 표준처럼 자리 잡기 때문이다. 경로 설계가 늦어질수록 이런 임시 선례가 기본값처럼 굳고, 나중에 세운 공식 경로가 오히려 예외 취급을 받게 된다.

에스컬레이션 경로를 설계할 때는 예외 처리 창구를 프로세스 오너와 분리해 두는 편이 실무적이다. 오너는 프로세스 전체의 방향을 책임지고, 예외 처리 창구는 개별 사례를 받아 즉시 판단하거나 상위로 올린다. 아래 조직도는 이 네 역할 — 프로세스 오너, 자동화 담당, 예외 처리 창구, 변경 승인권자 — 사이의 보고와 에스컬레이션 관계를 보여준다.

[그림 6-1] 네 역할 사이의 보고·에스컬레이션 경로



자료: 본문 정리

이 구조에서 예외 처리 창구는 반복되는 예외 유형을 기록해 자동화 담당에게 전달하는 역할도 겸한다. 같은 예외가 세 번 이상 반복되면 그것은 더 이상 예외가 아니라 재설계 대상이라는 신호다. 예외 처리 창구가 단순히 사례를 소화하는 역할에 머물면, 프로세스는 처음 설계했던 형태로 굳어진 채 현실과 점점 어긋난다.

변경은 누가 승인하는가

프로세스는 정지된 도면이 아니다. 취급 규정이 바뀌고, 조직 개편이 일어나고, 상위 시스템이 업데이트되면 자동화 로직도 함께 갱신되어야 한다. 그런데 이 갱신 책임의 소재가 불분명한 조직이 적지 않다. 업무 규정을 바꾼 부서는 자동화 로직까지 손볼 권한이나 지식이 없고, 자동화를 구축한 담당자는 업무 규정의 변경 여부를 통보받지 못한다.

변경 승인권자를 별도로 지정하는 이유가 여기에 있다. 이 역할은 업무 규정의 변경이 자동화 로직에 미치는 영향을 검토하고, 갱신 여부와 시점을 결정한다. 승인권자가 프로세스 오너와 같은 사람일 수도 있지만, 조직 규모가 커지면 이 둘을 분리해 변경 검토를 전담하는 편이 갱신 누락을 줄인다.

4

자동화 이후에 서야 하는 역할 — 오너 ·
자동화 담당 · 예외 창구 · 승인권자

변경 승인이 느슨하면 자동화 로직은 서서히 실제 업무 규정과 멀어진다. 반대로 승인 절차가 지나치게 무거우면 사소한 조정조차 병목에 걸려, 예외 처리 창구가 임시방편으로 우회하는 관행이 자리 잡는다. 두 극단 사이에서 승인 절차의 무게는 변경의 파급 범위에 따라 달라져야 한다 — 단일 조건문 수정과 전체 흐름의 재구성은 같은 절차를 거칠 필요가 없다.

파급 범위를 기준으로 삼으면 승인 절차는 자연스럽게 경량과 중량의 두 갈래로 나뉜다. 영향이 좁은 조정은 사후 보고로 충분하게 하고, 처리 경로 자체를 바꾸는 변경만 사전 검토를 거치게 하면 승인 부담과 갱신 누락 사이의 균형이 잡힌다. 경량 트랙의 기준은 문서로 못 박아 두어야 한다. 기준이 구두 합의로만 존재하면, 부담스러운 변경일수록 경량 트랙으로 분류하려는 유인이 생기기 때문이다.

거버넌스는 사후에 덧붙이는 항목이 아니다

프로세스 오너·자동화 담당·예외 처리 창구·변경 승인권자의 역할 분담은 자동화 구축이 끝난 뒤 정하는 부속 규정이 아니라, 재설계 단계에서부터 함께 설계해야 하는 항목이다. 역할이 나중에 배정되면 그사이 생긴 공백을 누군가 임시로 메우게 되고, 그 임시 조치가 관행으로 굳어진다.

네 역할이 문서로 남아 있으면 담당자가 바뀌어도 같은 질문 — 이 예외는 어디로 가는가, 이 변경은 누가 승인하는가 — 에 같은 방식으로 답할 수 있다. 결국 자동화가 오래 살아남는 조직과 금세 수작업으로 되돌아가는 조직의 차이는 기술의 정교함이 아니라 이 네 역할이 명확히 서 있는지에 있다. 도구는 규칙을 실행할 뿐이고, 규칙을 살아있게 유지하는 일은 여전히 사람의 몫이다.

재설계 프로젝트가 끝났다고 선언하는 시점을 자동화가 배포된 날이 아니라, 이 네 역할과 에스컬레이션 경로가 문서로 남고 관련자 모두가 자신의 책임 범위를 아는 날로 옮겨야 하는 이유도 여기에 있다. 그 날을 앞당기는 가장 확실한 방법은 역할 배정을 프로젝트 계획서의 마지막 장이 아니라 첫 장에 두는 것이다.

누가 오너이고 누가 예외를 받는지가 정해진 상태에서 구축이 진행된다면 문서화는 별도의 숙제가 아니라 구축 과정의 부산물이 되고, 구축 중의 설계 결정들도 운영을 맡을 사람의 눈으로 한 번 더 걸러진다.

자동화를 계속 살아 있게 만드는 것은 대단한 체계가 아니라 예외와 변경이 생길 때마다 같은 경로로 흐르게 하는 반복이며, 그 반복이 자리 잡은 조직에서 자동화는 특정 담당자의 프로젝트가 아니라 조직의 기본 동작이 된다. 예외 창구가 받는 사례의 수, 변경 승인이 처리되는 속도 같은 운영의 신호를 정기적으로 들여다보는 것만으로도, 자동화가 현실과 어긋나기 시작하는 시점을 놓치지 않을 수 있다.

07

재설계 이후의 조직 역량

자동화 재설계 프로젝트가 끝난 뒤에도 조직에 남아야 할 것 — 프로세스를 다시 그리는 습관 — 이 무엇이며 왜 일회성 프로젝트로 끝내면 안 되는지를 정리한다.

프로젝트로 끝나면 왜 다시 남는가
다시 반복할 수 있는 세 가지 역량
습관으로 남기 위한 방향

이 책은 진단에서 시작해 재설계 원칙, 기술 선택, 파일럿과 확산, 거버넌스까지 하나의 순서를 따라왔다. 그러나 이 순서를 한 번 완주했다고 해서 자동화가 끝나는 것은 아니다. 업무는 계속 변한다. 새로운 규정이 생기고, 조직 개편이 일어나고, 다루는 상품과 고객이 달라진다. 오늘 재설계한 프로세스는 그 순간의 최적일 뿐, 시간이 지나면 다시 낡는다. 프로젝트가 끝난 자리에 남아야 하는 것은 완성된 자동화 그 자체가 아니라, 이 순서를 다시 꺼내 쓸 수 있는 조직의 습관이다.

프로젝트로 끝나면 왜 다시 낡는가

1장에서 짚었던 실패 패턴을 다시 떠올려 보자. 도구를 먼저 들고 프로세스는 그대로 두었을 때 예외 처리가 폭증하고 병목이 다른 자리로 옮겨 갈 뿐이었던 그 패턴은, 사실 프로젝트가 일회성으로 끝날 때도 형태를 바꿔 다시 나타난다. 처음에는 제대로 재설계된 프로세스라도, 그 프로세스를 다시 들여다보고 고칠 사람과 절차가 조직에 남아 있지 않으면 결국 같은 자리로 돌아간다. 재설계는 결과물이 아니라 조직이 반복해서 수행할 수 있는 능력으로 자리 잡아야 실패 패턴의 재발을 막는다.

많은 조직이 자동화 프로젝트를 하나의 이벤트로 취급한다. 외부 컨설팅이나 태스크포스가 투입되어 진단하고, 원칙에 따라 재설계하고, 기술을 얹어 파일럿을 검증한 뒤 확산까지 마치면 프로젝트는 종료 보고서와 함께 닫힌다.

문제는 그다음이다. 프로젝트팀이 해산되고 나면 프로세스를 다시 그릴 줄 아는 사람도 함께 조직을 떠난다. 이후 벌어지는 작은 변화들 — 승인선 조정, 신규 시스템 연동, 예외 케이스 누적 — 은 임시방편으로 처리되고, 몇 년 뒤에는 처음 재설계 이전의 모습으로 서서히 되돌아간다. 앞서 다룬 진단·원칙·기술·실행·거버넌스는 한 번 쓰고 버리는 절차가 아니라, 조직이 스스로 반복할 수 있어야 값어치를 유지한다.

다시 반복할 수 있는 세 가지 역량

프로젝트의 성과를 습관으로 전환하려면 세 가지가 조직 안에 남아야 한다.

3

프로젝트가 끝난 뒤 남아야 할 역량 — 그리는 사람 · 재검토 주기 · 문서화 습관

첫째, 프로세스를 그릴 줄 아는 사람이다. 앞서 다룬 스위밍레인과 흐름도는 외부 전문가만 그릴 수 있는 그림이 아니다. 현업 실무자가 자기 업무를 레인과 단계로 나눠 그릴 수 있게 되면, 문제가 생겼을 때 매번 외부에 재진단을 의뢰할 필요가 없어진다. 이 역량은 한 번의 교육으로 끝나지 않고, 작은 프로세스 개선을 스스로 반복해 보는 경험을 통해서만 쌓인다.

둘째, 정기 재검토 주기다. 재설계는 완료 시점이 아니라 다음 재검토 시점을 정해두는 일이어야 한다. 재검토 없이 방치된 자동화는 업무가 바뀐 뒤에도 옛 규칙을 그대로 실행하다가, 예외가 쌓이고 나서야 문제로 드러난다.

재검토 주기를 정하는 기준

재검토 주기는 프로세스가 처리하는 업무의 변화 속도에 맞춘다. 규정이나 상품 구성이 자주 바뀌는 프로세스는 짧은 주기로, 안정적인 내부 절차는 긴 주기로 점검한다. 주기 자체보다 중요한 것은 담당자와 일정이 명시적으로 정해져 있는가다.

셋째, 자동화 로직의 문서화 습관이다. 6장에서 다룬 것처럼 자동화 로직이 한 사람의 암묵지로만 남으면 그 사람이 조직을 떠나는 순간 유지보수가 끊긴다. 무엇을 왜 그렇게 자동화했는지, 예외는 어떤 기준으로 분기하는지를 코드가 아닌 언어로 남겨두는 습관은, 다음번 재설계를 시작할 때 처음부터 다시 조사하지 않아도 되게 해준다.

이 세 가지를 다시 이 책이 다룬 순서와 겹쳐 보면, 프로젝트 단계와 지속 역량의 관계가 드러난다.

<표 7-1> 재설계 단계와 지속을 위한 조직 역량

이 책이 다룬 단계	반복하기 위해 남아야 할 역량
진단(as-is 매핑)	프로세스를 스스로 그릴 줄 아는 사람
재설계 원칙(ESIA)	재검토 시점마다 원칙을 다시 적용하는 습관
기술 선택과 실행	자동화 로직을 언어로 남기는 문서화 습관
거버넌스	오너십과 재검토 주기를 관리하는 체계

자료: 저자 정리

습관으로 남기 위한 방향

자동화 기술은 계속 발전하고, 앞으로도 더 정교한 도구들이 등장할 것이다. 그러나 어떤 도구가 나오든 그 도구를 엮기 전에 프로세스를 먼저 들여다보는 순서 자체는 바뀌지 않는다. 이 책이 제안한 진단-재설계-기술 선택-실행-거버넌스의 순서는 특정 기술 세대에 종속된 절차가 아니라, 조직이 스스로의 업무를 계속 다시 그릴 수 있게 하는 사고방식에 가깝다. 도구가 바뀌어도 이 순서를 꺼내 쓸 수 있는 조직은 새 기술을 프로세스 위에 신중하게 엮고, 그렇지 못한 조직은 다음번에도 도구부터 들이는 실수를 반복한다.

재설계 프로젝트가 남겨야 할 최종 산출물은 자동화된 화면 몇 개가 아니다. 프로젝트팀이 해산되고 시간이 흘러도, 누군가 프로세스를 다시 펼쳐 그리고 왜 이렇게 되어 있는지 설명할 수 있고, 정해진 시점에 다시 점검하는 자리가 열리며, 그 논의를 이어받을 기록이 남아 있는 상태—그것이 이 책이 처음부터 끝까지 제안해 온 순서가 실제로 조직에 뿌리내렸다는 증거다. 재설계는 한 번의 프로젝트가 아니라, 다음번에도 같은 순서를 스스로 꺼내 쓸 수 있는 조직의 습관으로 남아야 완성된다.

